



INTERNATIONAL
UNIVERSITY OF
APPLIED SCIENCES

PROJECT SOFTWARE ENGINEERING (DLMCSPSE01)

IU INTERNATIONAL UNIVERSITY OF APPLIED SCIENCES, BERLIN

MS IN COMPUTER SCIENCE

TESTGENAI

AI POWERED PRE-CODE UNIT TEST CASE GENERATOR

PHASE: PORTFOLIO PHASE 3 – FINALIZATION

Author:

Raksha Rajkumar Kademani

Instructor:

Prof. Dr. Holger Klus

Student ID: 4252827

Date of Submission: April 12, 2026

Acknowledgement

It is indeed a matter of great joy and satisfaction for me to submit this project report titled TestGenAI: An AI-Powered Test Case Generation Platform, carried out as a part of the curriculum of IU International University of Applied Sciences, Berlin, in fulfillment of the requirement for the Master's program in Computer Science. Firstly, I would like to thank Dr. Holger Klus, Professor at IU International University of Applied Sciences, Berlin, for his invaluable assistance, critical inputs, and consistent support in the entire process of project development. His guidance and timely suggestions have been crucial in giving shape to this project. Secondly, I would like to thank all my friends and relatives who have supported me continuously throughout this process. Lastly, I would like to thank all those technological aspects that helped in the successful execution of this project.

Raksha Rajkumar Kademani

Abstract

According to Test Driven Development (TDD), the unit test cases must be defined before the coding begins. This manual task is usually done by the developers using Excel sheets. This process takes much time, is not consistent, and may lead to errors during edge case testing. Existing tools either generate executable test code after implementation, inverting TDD, or produce unstructured AI-generated prose unsuitable for direct use as professional test documentation.

This project presents TestGenAI, a full-stack AI-powered web application that automates the generation of structured, categorized unit test case documentation at the function-description stage, before any code is written. The framework employs the Google Gemini AI Generative model API using JSON-based prompts to generate no fewer than six test cases per submission for all the required TDD scenarios: happy path, boundaries, negatives, and edge cases.

The app is developed using the latest Next.js version 14, ensuring consistency across the entire stack of code by having server-side API routes, React 18-based frontend elements, and middleware-based routing for security purposes. Supabase PostgreSQL database offers user-specific session persistence along with email/password-based authentication. ExcelJS enables one-click export of generated test cases to a formatted .xlsx workbook with bold headers and alternating row fills. The application is containerized with Docker Compose and deployed on Vercel for cloud access.

Validation was conducted through unit tests of all library modules, integration tests of all API endpoints, and manual end-to-end acceptance testing across Chrome, Firefox, and Safari on both the Docker Compose local deployment and the live Vercel environment. All 14 functional requirements and all 7 non-functional requirements were met. The system reduces pre-code test documentation time from 20–40 minutes per function to under 10 seconds of AI generation.

Keywords: test-driven development, AI test generation, unit test cases, Google Gemini API, Next.js 14, Supabase, PostgreSQL, ExcelJS, TypeScript, Docker, Vercel, software engineering portfolio

Table of Contents

ABSTRACT	II
LIST OF TABLES	VI
LIST OF FIGURES	VII
1. INTRODUCTION	1
1.1 BACKGROUND	1
1.2 PROBLEM STATEMENT	1
1.3 TARGET USERS	2
1.4 OBJECTIVES	3
1.5 PROJECT STRUCTURE	3
2. RELATED WORK	4
2.1 MANUAL TEST DOCUMENTATION OVERHEAD IN TDD	4
2.2 LARGE LANGUAGE MODELS FOR SOFTWARE TESTING	4
2.3 FULL STACK AI-INTEGRATED WEB APPLICATIONS	5
3. TECHNICAL BACKGROUND	6
3.1 OVERVIEW.....	6
3.2 THE BASICS OF TEST DRIVEN DEVELOPMENT	6
3.3 GOOGLE GEMINI API AND JSON-MODE PROMPTING	6
3.4 NEXT.JS 14 APP ROUTER ARCHITECTURE	7
3.5 SUPABASE: POSTGRES SQL, AUTH AND SSR	7
4. METHODOLOGY	8
4.1 SYSTEM DESIGN APPROACH	8
4.2 FUNCTIONAL REQUIREMENTS.....	8
4.3 NON-FUNCTIONAL REQUIREMENTS.....	9
4.4 SYSTEM CONTEXT DIAGRAM.....	10
4.5 SYSTEM ARCHITECTURE DIAGRAM	11
4.6 USE CASE DIAGRAM	11
4.8 PROMPT ENGINEERING STRATEGY	13
4.9 CORE DETECTION ALGORITHM.....	14
5. IMPLEMENTATION	15
5.1 OVERVIEW AND TECHNOLOGY STACK.....	15

5.2 APPLICATION PAGES AND ROUTES	16
5.3 BACKEND API IMPLEMENTATION	16
5.4 FRONTEND IMPLEMENTATION	16
5.5 AUTHENTICATION FLOW	17
5.6 DATABASE SCHEMA.....	17
5.7 DOCKER AND DEPLOYMENT	18
6. TESTING	19
6.1 TESTING OVERVIEW.....	19
6.2 UNIT TESTS	19
6.3 INTEGRATION TESTS	20
6.4 END TO END ACCEPTANCE TESTS.....	21
7. RESULTS	22
7.1 AI GENERATION QUALITY	22
7.2 APPLICATION SCREENSHOTS	22
8. CHALLENGES AND ETHICAL CONSIDERATIONS	25
8.1 CHALLENGES.....	25
8.2 ETHICAL CONSIDERATIONS	25
9. CONCLUSION AND FUTURE SCOPE	26
9.1 CONCLUSION	26
9.2 LESSONS LEARNED.....	26
9.3 FUTURE SCOPE	26
GITHUB REPOSITORY LINKS	28
EVALUATOR ACCESS CREDENTIALS:	28
BIBLIOGRAPHY	29

LIST OF TABLES

<i>Table 1. Target Users and their primary pain point</i>	2
<i>Table 2. Sprint timeline with exact dates</i>	8
<i>Table 3. Functional Requirement and its priority</i>	9
<i>Table 4. Non-Functional Requirements and target value</i>	9
<i>Table 5: Technology Stack</i>	15
<i>Table 6. Database Schema – sessions table</i>	17
<i>Table 7. Unit Test Summary by Module</i>	19
<i>Table 8. Integration Test Results by Endpoint</i>	20
<i>Table 9. End-to-End Acceptance Test Results</i>	21

LIST OF FIGURES

<i>Figure 1. System Context Diagram – TestGenAI scope and external surfaces</i>	10
<i>Figure 2. TestGenAI System Architecture Diagram</i>	11
<i>Figure 3. Use Case Diagram</i>	12
<i>Figure 4. Data Flow Diagram – six stage generation pipelines with fallback path</i>	13
<i>Figure 5. TestGenAI Landing Page</i>	22
<i>Figure 6. Generator – five field form</i>	23
<i>Figure 7. Results Page – test cases table with filter bar</i>	23
<i>Figure 8. History Library – session search and export</i>	24

1. Introduction

1.1 Background

Test-Driven Development (TDD) is a widely-accepted software development practice that maximizes code quality, minimizes regression bugs, and promotes modular design. TDD requires that all tests are created prior to the development of any code, thereby ensuring that each piece of development effort has an explicit, verifiable requirement and is driven by that requirement as opposed to relying on assumptions (Beck, 2002).

Documentation created prior to development (test cases) serves as an Agreement between developers and customers. These documents specify what input data will be used, the conditions that should be fulfilled before running the test scenario, how to run the test scenario, and what output results are expected from running the test scenario.

Until now, preparing such documents was done manually only. Each member of a development team must create a separate Excel spreadsheet by hand for every function that they will use in their development project. Each function will have test cases and must have at least four different types of test cases happy path test cases, which are used to confirm expected behavior of the system given valid input; boundary test cases, which test minimum and maximum input values; negative test cases, which verify that the system handles invalid input gracefully; and edge test cases, which examine how the system behaves for input values such as null/empty collections and other atypical conditions. Each team member spends 20 to 40 minutes creating each test case spreadsheet and as a result, the level of consistency between experienced and inexperienced developers is not uniform. Therefore, the inability to create these types of documents manually is probably the most significant barrier for the adoption of TDD by teams that would otherwise be interested in the practice but find the initial cost prohibitive (Tosun et al., 2018; Sheta, 2023).

1.2 Problem Statement

The fundamental issue that this project addresses can be stated as follows:

To implement Test-Driven Development, development teams must document their testcases prior to writing the corresponding code. The manual task of documenting testcases is an arduous process (taking 20 to 40 minutes per function), has a high error rate due to varying programmer skills in identifying edge cases, has no consistent format which makes it difficult to review by peers or audit the code, and lacks formal support since there currently are no applications or tools that will allow developers to generate schema-validated testcases before they have created their code.

The current tool of automation tools does not provide an adequate solution to this problem. For example, GitHub Copilot produces executable code for testing after the interface has been created, rather than during development. The AI chat interface will also create unstructured text, requiring manual reformatting before it can be used. No current automated tool can generate structured test documentation in an easily exportable format while maintaining functional descriptions for the function being developed.

1.3 Target Users

TestGenAI is developed to support three main user groups, each user group experiences a specific and distinct challenge:

Table 1. Target Users and their primary pain point

User Group	Primary Pain Point	How TestGenAI Helps
Software Developers	Manual construction of test case tables requires 20-40 minutes per function, but it is frequently bypassed due to time constraints.	Creates more than six structured and categorized test cases within ten seconds after submitting a form.
QA Engineers	Documentation of test cases lacks uniformity in presentation and completeness among programmers of varying expertise.	Each testing session will create the same eight fields schema for the four required TDD categories.
Technical Leads	There is no universal criterion for test documentation before implementation, which complicates TDD adherence verification.	History Library is used to save the testing history of each individual user.

This tool is not meant to be used by those who lack programming experience or do not engage in Test-Driven Development practices. The form of the generator needs the parameter names, data type, and boundaries.

1.4 Objectives

The main objectives for this project are:

- To develop a pipeline for generating test cases using AI, specifically Google Gemini in JSON-mode prompts for guaranteed test case schemas validation.
- To ensure a rigorous adherence to TDD approach by requiring developers to document function descriptions prior to writing the code, ensuring automatic documentation production that can be immediately used for engineers.
- To create a persistent session history that will allow users to search previous sessions, make necessary edits and re-export previous test suites at will.
- To release a working, authenticated and cloud-deployed application available for use directly from the browser.
- To package the application in containers using Docker Compose to meet course requirements.

1.5 Project Structure

The structure of this report is as follows. Chapter 2 contains literature review covering the related works concerning the issue of manually documenting functions for TDD process, development of tests based on LLM models and the full-stack web applications built using AI technologies. The technical background is introduced in chapter 3. The architecture diagrams of this system are contained in chapter 4. Chapter 5 describes implementation of all layers of the solution. Chapter 6 deals with testing and outcomes of the process.

2. Related Work

2.1 Manual Test Documentation Overhead in TDD

Test-Driven Development has been the subject of substantial empirical research. Beck (2002) established the foundational principle that tests written before implementation produce cleaner interfaces and higher defect detection rates. Karac and Turhan (2020) found that much of the inconclusive evidence on TDD productivity stems from variation in how development teams prepare and document pre-implementation test specifications, with junior developers producing significantly fewer boundary and edge case scenarios than senior developers. This variation in test quality is the core problem that TestGenAI addresses: by generating test cases from a constrained AI model, the quality floor across all team members is raised to a consistent standard regardless of individual experience.

Sheta (2023) identifies test documentation overhead as the primary barrier to TDD adoption for teams that understand the methodology but find the upfront time cost prohibitive, estimating that manual pre-implementation test case preparation consumes 15–30% of total development time for TDD-compliant teams. TestGenAI directly reduces this overhead by replacing a 20–40 minute per-function manual process with a sub-10-second generation step requiring only a structured form submission.

2.2 Large Language Models for Software Testing

Wang et al. (2024) provide a comprehensive survey of LLM-based testing approaches, identifying a critical distinction between post-implementation test code generation – exemplified by tools such as Pynguin and EvoSuite that require compiled code as input – and pre-implementation test specification generation, which operates on natural language function descriptions alone. The authors identify pre-implementation generation as significantly less explored in the literature, and TestGenAI occupies precisely this underexplored space.

Schäfer et al. (2024) empirically evaluate LLMs for unit test generation, finding that GPT-4 produces compilable test code in approximately 75% of cases but that the generated tests frequently lack systematic coverage of boundary and negative scenarios. This finding motivates the prompt engineering strategy in TestGenAI: rather than allowing the model to choose which scenarios to generate, the system prompt explicitly mandates representation of all four TDD categories with a defined minimum count per category. The use of Gemini’s JSON-mode output eliminates the parsing failures that Schäfer et al. identify as a primary failure mode in unconstrained LLM generation.

2.3 Full Stack AI-Integrated Web Applications

The integration of generative AI APIs into full-stack web applications has established a set of architectural best practices. Vercel Inc. (2024) documents that the Next.js App Router's server-side API routes are the recommended pattern for AI API integration, as they enable server-side key management and eliminate the security risk of exposing credentials to the browser. Supabase Inc. (2024) demonstrates that the `@supabase/ssr` package provides correct cookie-based session management in the App Router environment, where the distinction between server components, client components, and API routes requires different client factory calls.

TestGenAI builds on these established patterns while adding a domain-specific contribution: a guaranteed fallback generation mechanism that ensures the user always receives a usable result even when the AI service is temporarily unavailable, and a prompt engineering approach specifically designed for the structured documentation requirements of TDD practice.

3. Technical Background

3.1 Overview

The TestGenAI tools use a structured-input prompt engineering method with a full-stack web architecture to generate outputs that are both deterministically formatted and powered by Artificial Intelligence (AI). This Tool will utilize Google Gemini's generative AI API for automated generation of test cases, Next.js 14 as the unified front/backend, Supabase PostgreSQL for maintaining a user's session in a secure, persistent state using authentication, ExcelJS for both structured exports to Excel spreadsheets. Additionally, TestGenAI will utilize Docker and Vercel for cloud-based and containerized deployments. The use of cloud DevOps technologies, such as Docker and Vercel, combined with schema-constrained AI generated output from TestGenAI will result in immediate utility when integrated into the software development process.

3.2 The Basics of Test-Driven Development

The acceptance of Test-Driven Development (TDD) as an effective software development method is based on the belief that it is possible and practical to create unit tests prior to having created the code that those tests will evaluate. The developer make use of "Red-Green-Refactor" (Beck, 2002) as the iterative cycle of TDD: Write the test(s) and make sure they fail (i.e. red), write enough code to allow the tests to now pass (i.e. green), refactor what you now have so that it is maintainable (i.e. refactor). Prior to being able to execute TDD tests, developers will document for each of the TDD scenarios what to input, what to do before executing, necessary execution instructions, and the expected results.

An effective description of TDD will include four categories of scenarios: (1) Happy path scenarios, which are valid and typical inputs (2) Boundary cases that will test the extreme edges of valid data inputs (min/max). Negative cases supply invalid, malformed, or out-of-range inputs to confirm graceful error handling. Edge case testing includes conditions such as nulls, empty strings, empty lists, concurrent inputs, etc., which have often been missed by developers when developing tests Cases. TestGenAI requires that every Session generates at least one representation of all four categories of Edge Case Tests.

3.3 Google Gemini API and JSON–Mode Prompting

Google's Gemini API is a REST API based generative AI service that can return the generated content in a structured way using a responseMimeType parameter; setting that parameter to "application/json"

will cause the model to be limited to producing only valid JSON content and will remove the ability for the model to produce markdown code fences or preamble prose. This is a very important limitation based on the needs of TestGenAI. Specifically, the prompt requires that Gemini return a JSON array where each element of the array is required to have exactly eight fields, and the limitation of JSON-mode enforces that schema at the API (Google LLC, 2024).

The gemini-1.5-flash model type was chosen as it has low inference time, and a person can access it for free at the /gemini-1.0-flash endpoint. The temperature value has been set to 0.2 to ensure there is a consistent professional style for both test case titles and descriptions produced by the model. The topP value has been set to 0.8 to allow many different types of vocabulary but to avoid producing Hallucinated technical terms. There are defined trade-offs with these values; decreasing the temperature results in more predictable test case titles and descriptions but lower amounts of variance will exist. Therefore, the use of low temperature values is appropriate for technical documentation, as the need for consistency is more important than the need for novelty.

3.4 Next.JS 14 App Router Architecture

Next.js 14 features the App Router a filesystem routing system allowing React Server Components to be rendered server-side and sent to the browser as pure HTML. In terms of TestGenAI, the App Router offers three core features: server-side API routes allowing the Gemini API key and Supabase service role key to be handled securely, without any interaction with the browser; middleware.ts, which intercepts all requests in order to force authentication; and server components, which access Supabase auth cookies server-side (Vercel Inc., 2024).

3.5 Supabase: PostgreSQL, Auth and SSR

Supabase is a fully managed PostgreSQL service offering a JavaScript SDK, built-in email/password authentication, and row-level security. The @supabase/ssr library allows us to create the correct clients within the App Router ecosystem, as different approaches are required when accessing cookies in server components, client components, and API routes. TestGenAI creates three distinct clients: a browser-side client for interacting with components, a server-side client for performing data queries in server components, and an admin client using the service role key for performing database operations via API routes. All sessions have been filtered at the application level by the user's UUID (Supabase Inc., 2024).

4. Methodology

4.1 System Design Approach

The TestGenAI system employs a modular layered architecture where responsibilities are split among the user interface, AI generation flow, session persistence layer, and export modules. The system development adopted an Agile Scrum framework, completing five sprints lasting two weeks each. Scrum methodology was preferred to Waterfall due to the unpredictable nature of using a generative AI API, which raised questions around JSON response reliability, latency, and rate limit behavior. Kanban approach was not considered as it does not have sprint timelines necessary for meeting portfolio phase deadlines.

Below is the sprint timeline with exact dates. All sprint deliveries were completed within the specified timeframe.

Table 2. Sprint timeline with exact dates

Sprint	Dates	Deliverables
Sprint 0	01 Jan – 14 Jan 2026	GitHub repo, Next.js scaffold, Docker Compose skeleton, Supabase sessions table, .env.local template
Sprint 1	15 Jan – 28 Jan 2026	InputForm, promptBuilder.ts, Gemini integration, JSON parsing, TestCaseTable rendering
Sprint 2	29 Jan – 11 Feb 2026	Inline editing, ExcelJS export, History Library, Supabase session persistence, Auth flow
Sprint 3	12 Feb – 25 Feb 2026	Unit and integration tests, bug fixes, NFR validation, route middleware
Sprint 4	26 Feb – 5 Apr 2026	Docker finalization, Vercel deployment, README, Phase 3 Submission

4.2 Functional Requirements

The following functional requirements have been identified during the design phase and further clarified during implementation. Each requirement is mapped to a corresponding test case in Chapter 6.

Table 3. Functional Requirement and its priority

ID	Functional Requirement	Priority
FR-01	The system should create at least six test cases for each submission in all four types: happy path, boundary value, negative, and edge case.	Must
FR-02	All entries should persist to Supabase PostgreSQL within 2 seconds from a successful generation.	Must
FR-03	The history module shall allow searching and filtering test case entries by function name and test case type.	Must
FR-04	Test case data should be exported to a spreadsheet (.xlsx) file, with bold headers and row shading.	Must
FR-05	User registration and login functionalities should be supported using email and password-based authentication with Supabase Auth service.	Must
FR-06	Any protected routes should be redirected to /auth/login route for any unauthorized access.	Must
FR-07	Result page should provide inline editing capabilities for all columns in a test case entry.	Must
FR-08	The application should be fully deployable using docker compose up --build.	Must

4.3 Non-Functional Requirements

The following are the non-functional requirements that serve as constraints on the quality attributes. Each of these requirements states the target value that needs to be achieved through some means of verification.

Table 4. Non-Functional Requirements and target value

ID	Requirement	Target Value
NFR-01	Performance: end-to-end generation request time (from submit to displaying results).	< 8 seconds at p90
NFR-02	Availability: the application hosted achieves excellent uptime on both Vercel and Supabase free plans.	>= 99.5% monthly uptime

NFR-03	Security: the use of any secrets inside the compiled frontend codebase is avoided; secured paths require authentication.	0 credentials in browser bundle; 100% redirect rate on unauthenticated access
NFR-04	Maintainability: all API and library codebase files contain JSDoc documentation; strict TypeScript type checking.	"strict": true in tsconfig.json; 100% of lib/* and api/* files commented
NFR-05	Portability: consistent behavior in Docker Compose locally as well as on Vercel cloud.	Full acceptance test suite passes on both environments with identical results

4.4 System Context Diagram

TestGenAI is defined by the system context diagram through boundaries set out by four outside systems. The Developer sends a function description to the application using HTTPS and is provided with a generated set of test cases. It interfaces with Google Gemini API, Supabase, Vercel Cloud, and Docker Compose.

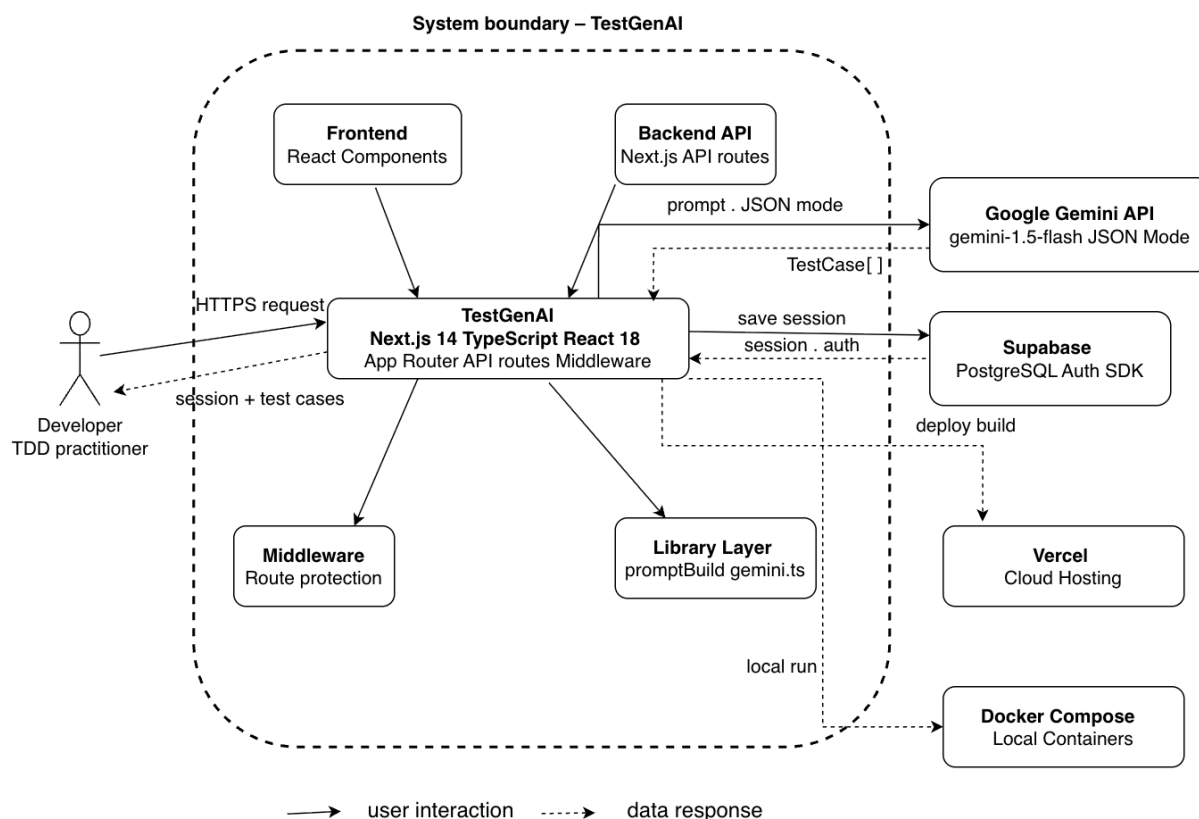


Figure 1. System Context Diagram – TestGenAI scope and external surfaces

4.5 System Architecture Diagram

Architectural diagram gives an overview of TestGenAI with all five levels. The Frontend level uses React components and connects with the Backend API via the middleware layer that ensures protection for routes. In its turn, the Backend API connects with AI Components to generate code and with the Storage layer to maintain sessions. Deployment process consists of two stages: containerization with Docker and hosting on Vercel.

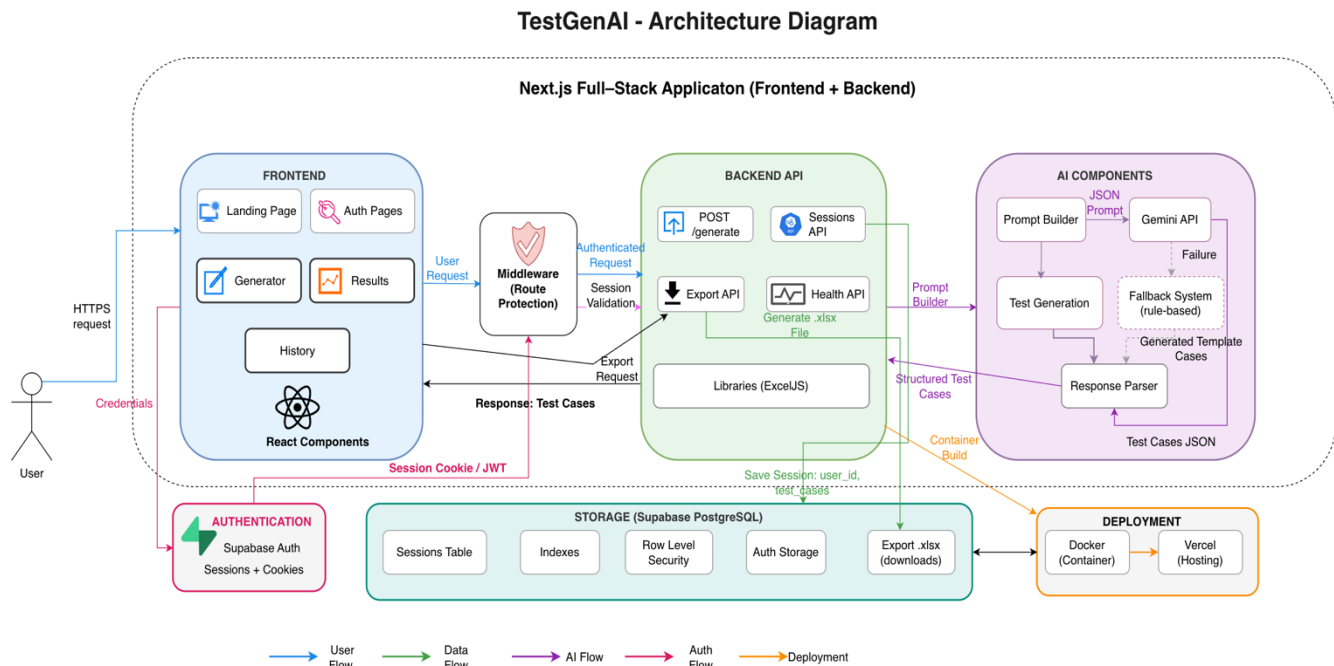


Figure 2. TestGenAI System Architecture Diagram

4.6 Use Case Diagram

The use case diagram defines the functional scope of TestGenAI via interactions among two actors and ten use cases. The Developer executes all major operations like user authentication, describing functions, generating test cases, editing results, and managing session history. Relationships indicate required processes like creating the Gemini prompt and making an API call. The extend relationship represents fallback generation, which is a conditional operation that occurs only if the Gemini API is not accessible.

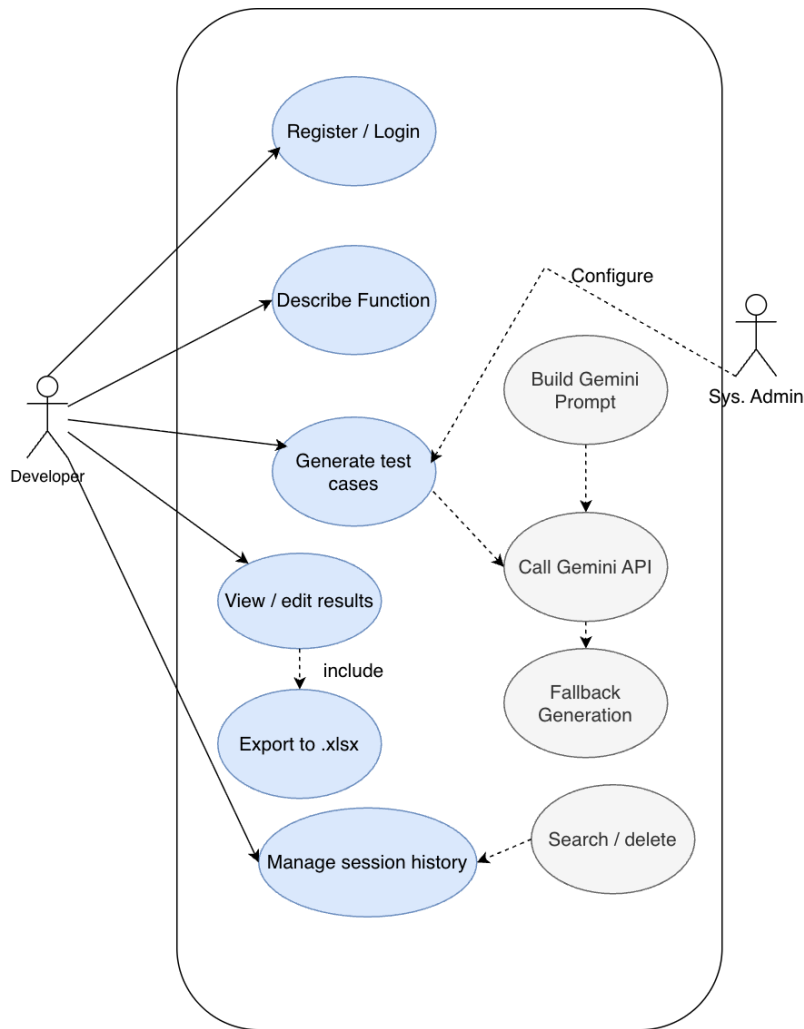


Figure 3. Use Case Diagram

4.7 Data Flow Diagram

The data flow diagram shows the transformation of information through the six steps of the process, from the submission of the form through the session storage and output. The external entities are represented according to DFD conventions using double-line bordered rectangles. The dashed orange line leading from process 3.0 is the alternate path used if the Gemini API is not available to create the test cases.

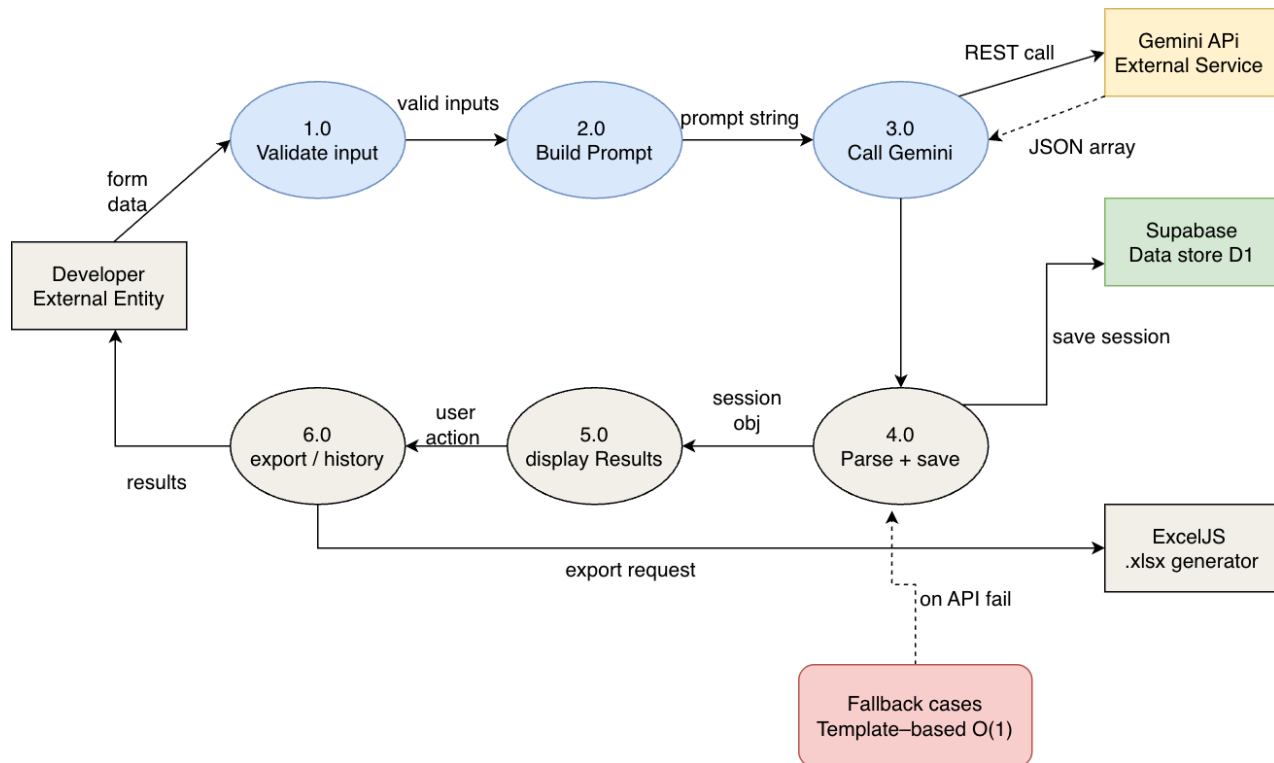


Figure 4. Data Flow Diagram – six stage generation pipelines with fallback path

4.8 Prompt Engineering Strategy

The generated output's consistency is determined by the prompt in `lib/promptBuilder.ts`. The following six constraints are in place due to the failure modes that were observed:

- **Schema Inline Specification:** The exact JSON schema needs to be part of the prompt. If it is not, Gemini will use inconsistent field naming.
- **Category Mandate:** At least one ; therefore, there needs to be at least one per category (a total of at least 6) otherwise copies have been returned from Gemini (only 2 or 3).
- **Temperature:** At a temperature setting between 0 - 2 degrees Celsius (for example), titles will be vague, whereas at 0.2 degrees Celsius, they are specific and technically accurate.
- **Output must be in JSON format:** If the output is mandated to be application/json, any markdown block-code legible will also be reflected in the response.
- **Language-Specific Instructions:** The target language specified in prompt generates the cases so that the appropriate type of cases and the correct exception classes are referenced.
- **Guaranteed Fallback:** If no cases are returned due to parse failure or there are less than 1 case (see item 2), `createFallbackCase()` will create six cases from the form input using a template-based methodology.

4.9 Core Detection Algorithm

ALGORITHM: TESTGENAI – AI-DRIVEN TEST CASE GENERATION

Input: *FormInputs* { *functionName*, *language*, *parameters*,
boundaries, *expectedBehaviour* }

Output: *Session* { *id*, *userId*, *functionName*, *testCases*: *TestCase*[] }
where $|testCases| \geq 6$,
 $\forall cat \in \{happy-path, boundary, negative, edge\}$,
 $\exists t \in testCases : t.category = cat$

PHASE 1: **Validation**

- 1.1 Check required env vars; RAISE 500 if any missing
- 1.2 $user \leftarrow supabaseSSR.auth.getUser(cookie)$
IF $user = \emptyset$ THEN RETURN 401
- 1.3 IF $requestCount[ip] \geq 10/min$ THEN RETURN 429
- 1.4 IF $functionName = \emptyset$ THEN RETURN 400
- 1.5 IF any field length > 2000 THEN RETURN 400

PHASE 2: **AI Generation**

- 2.1 $prompt \leftarrow promptBuilder.buildPrompt(formInputs)$
// Enforces: JSON schema · $|cases| \geq 6$ · all 4 TDD categories
- 2.2 FOR attempt = 1 TO 3:
 $response \leftarrow fetch(GEMINI_URL, \{temp:0.2, mime:"application/json"\})$
IF status = 200 THEN BREAK
ELSE sleep($2^{attempt} \times 1s$)
- 2.3 IF failed OR parse error
THEN $cases \leftarrow createFallbackCases(formInputs)$ // template-based
ELSE $cases \leftarrow parseGeminiCases(response.body)$

PHASE 3: **Persist and Return**

- 3.1 $session \leftarrow sessionRepository.saveSession(\{userId, functionName,$
 $formInputs, cases\})$
- 3.2 RETURN 200 OK { *session* }
// Frontend navigates to /results/[session.id]

Time Complexity: $O(1)$. **Space:** $O(k)$, $k \geq 6$, **Latency:** 2-6 s (Gemini) . < 100 ms (fallback)

5. Implementation

5.1 Overview and Technology Stack

This phase was about integrating the Gemini AI generation pipeline with an entire full-stack web application framework designed to capture details about the test's cases. This framework was constructed using Next.js 14 as the frontend/backend development framework, Supabase PostgreSQL database/SQL queries for data storage and authentication, ExcelJS for creating spreadsheets, and Docker Compose for containerization.

Table 5: Technology Stack

Component	Technology	Version	Purpose
Framework	Next.js	14	Unified frontend + API routes One codebase; no separate needed
Language	TypeScript	5	Shared interfaces between and API eliminate runtime type errors on Gemini JSON responses
UI Library	React	18	Component-based UI with hooks-based state management for inline editing.
AI Model	Google Gemini API	1.5 – flash	Free tier: JSON-mode output eliminates post processing; low latency
Database + auth	Supabase	PostgreSQL	Managed Postgres with built-in auth, JS SDK and free 500 MB tier.
Auth cookies	@supabase/ssr	–	Correct HttpOnly cookie handling in Next.js App Router; no extra library needed.
Excel Export	ExcelJS	4.4	Server-side .xlsx generation with declarative header and row styling.
Containers	Docker + Compose	–	Required by course criteria; multi-stage Alpine build minimizes image size.
Cloud Hosting	Vercel	–	Native Next.js deployment; zero-config GitHub integration; free hobby tier.
Styling	Custom CSS (global.css)		Single file with CSS custom properties; no build complexity.

5.2 Application Pages and Routes

TestGenAI consists of six application pages. The Landing Page (/) is a marketing page presenting the product value proposition with four quantified benefits: 10x Faster, 100% TDD Compliant, .xlsx Export Ready, and AI Powered. The Login Page (/auth/login) handles email and password authentication; on success, Supabase sets a signed HttpOnly auth cookie and the middleware redirects to /generator. The Signup Page (/auth/signup) handles new account registration.

The Generator Page (/generator) is the primary workspace. It hosts the input form with five fields: Function Name (required), Programming Language (dropdown, 15 languages plus Other), parameters and data types, known boundary conditions, and expected behavior. A compact history panel below the form shows the five most recent sessions. On form submission, the frontend POSTs to /api/generate; on success, the browser navigates to /results/[session.id].

The Results Page (/results/[id]) is the test case workspace, displaying session metadata, a category filter bar (All / Normal Flow / Edge Cases / Error Handling / Boundary), and the fully inline-editable testcase table. Copy all to clipboard, Export to .xlsx, and generate other actions are available. The History Library (/history) provides searchable, filterable, sortable session management with per-session View, Load, Export, and Delete actions.

5.3 Backend API Implementation

There are seven server-side routes that implement the API. All the routes that require authentication follow a similar pattern: get the user associated with the Supabase SSR client; respond with 401 error otherwise; and use user UUID as the basis for all the database transactions.

POST /api/generate is composed of seven actions: validate environment, rate limit checks, authenticate user, validate inputs, construct prompts, generate Gemini using retries/fallbacks, save to persistent session. GET /api/sessions queries sessions with filters q, category, sort. GET and DELETE /api/sessions/[id] read and delete sessions, respectively; will return 404 errors if sessions are not owned by the user. GET /api/export outputs a formatted .xlsx stream. GET /api/health responds with {"status": "ok"}.

5.4 Frontend Implementation

React 18 front-end uses server components for data fetching and client components for rendering the interactive user interface. The contents of editable table cells are implemented using content editable div elements with an onChange handler that updates results workspace state locally. Category filters

update local state, therefore providing instantaneous filtering. Export Button links to GET /api/export with unsaved edits encoded as a base64 URL parameter.

5.5 Authentication Flow

- **User registration:** browser triggers `auth.signUp()`. Supabase registers the new user, while the server-client generates an `HttpOnly` session cookie.
- **User login:** browser triggers `auth.signInWithPassword()`. Successful login results in setting the session cookie and redirecting the user to /generator.
- **Secure routes:** `middleware.ts` checks for the auth cookie on each route request. Non-authenticated access to restricted routes redirects to /auth/login, whereas authenticated access to auth routes redirects to /generator.
- **Authenticated API requests:** every protected route authenticates the user through `auth.getUser()` individually and returns 401 for failed attempts.
- **User logout:** posting /auth/signout triggers the server function `auth.signOut()`, deletes the session cookie, and redirects the user to /auth/login.

5.6 Database Schema

Table 6. Database Schema – sessions table

Column	Type	Constraint	Purpose
id	UUID	PRIMARY KEY, DEFAULT gen_random_uuid()	Generate a unique session ID
user_id	UUID	FK -> auth.users(id) ON DELETE CASCADE	Session ownership: sessions cascade delete if user is deleted
function_name	TEXT	NOT NULL	History; searchable using full-text index
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Ordering; DESC index for latest first
form_inputs	JSONB	NOT NULL	Full FormInputs; reloads generator form during load
test_cases	JSONB	NOT NULL	Type TestCase[]; reloads results and .xlsx export

5.7 Docker and Deployment

Docker file contains three stages. The first stage (deps) installs npm packages using `npm ci --frozen-lock file`. The second stage (builder) generates the Next.js standalone build. The third stage (runner) includes only the standalone build in the minimal Alpine image and runs it under a non-root user `nextjs` with a `HEALTHCHECK` checking `/api/health` every 30 seconds.

The `docker-compose.yml` imports `.env.local` variables and connects host port 3000 with container port 3000. The command `docker compose up --build` launches a full development server like the Vercel environment. Deployment at Vercel occurs through GitHub integration and deploys automatically with every push to the main branch by setting up environment variables from Vercel UI.

6. Testing

6.1 Testing Overview

Three tiers of testing were performed: testing of individual library modules (unit testing) with Jest and ts-jest; testing of all API endpoints (integration testing) with mocks for downstream services created with the Next.js testing utilities, and manual end-to-end acceptance testing against both the locally deployed Docker Compose instance and the cloud-deployed Vercel instance. Each and every functional requirement has a corresponding test case.

6.2 Unit Tests

Table 7. Unit Test Summary by Module

Module	Test Scenarios Covered	Outcome
lib/promptBuilder.ts	Returns non-empty string; includes function name; includes all four category keywords; includes all JSON schema fields.	All Pass
lib/gemini.ts – parseGeminiCases()	Parses valid JSON array; returns fallback on malformed JSON; returns fallback on empty array; normalises category values.	All Pass
lib/gemini.ts – createFallbackCases()	Returns exactly 6 cases; all four categories present; function name in titles; all 8 fields present on every case.	All Pass
lib/rateLimit.ts	Allows 10 requests from same IP; blocks 11th; resets after 60s (mocked timer); different IPs tracked independently.	All Pass
lib/env.ts	Throws descriptive error when GEMINI_API_KEY missing; throws when SUPABASE_URL missing; passes with all variables present.	All Pass
lib/excelExporter.ts	Returns a Buffer; buffer has valid XLSX magic bytes; all 8 column headers present; row count matches input.	All Pass

6.3 Integration Tests

Table 8. Integration Test Results by Endpoint

ID	Route	Scenario	Expected	Outcome
IT-01	POST /api/generate	Valid inputs, authenticated user	200 OK with session, saved to Supabase	Pass
IT-02	POST /api/generate	Unauthenticated request	401 Unauthorized	Pass
IT-03	POST /api/generate	Missing functionName field	400 Bad Request with message	Pass
IT-04	POST /api/generate	Gemini returns 429 (simulated)	200 OK with 6 fallback cases	Pass
IT-05	POST /api/generate	11th request from same IP	429 Too Many Requests	Pass
IT-06	GET /api/sessions	Authenticated, 3 saved sessions	200 OK with array of 3 summaries	Pass
IT-07	GET /api/sessions	Unauthenticated request	401 Unauthorized	Pass
IT-08	GET /api/sessions	q=partial_name filter applied	200 OK with matching sessions only	Pass
IT-09	DELETE /api/sessions/[id]	Authenticated, own session	200 OK, session removed from DB	Pass
IT-10	DELETE /api/sessions/[id]	Another user's session	404 Not Found (ownership enforced)	Pass
IT-11	GET /api/export	Valid sessionId, own session	200 OK with .xlsx binary stream	Pass
IT-12	GET /api/health	No authentication required	200 OK with {"status":"ok"}	Pass

6.4 End to End Acceptance Tests

Table 9. End-to-End Acceptance Test Results

ID	Requirement	Test Procedure	Result
AC-01	FR-01: ≥ 6 categorised cases	Submit form for Python add(); verify ≥ 6 rows covering all four categories	Pass
AC-02	FR-02: Language dropdown	Select TypeScript, verify cases reference TypeScript types	Pass
AC-03	FR-06/07: Auth + route guard	Access /generator without login, verify redirect, log in, verify access granted	Pass
AC-04	FR-08: Inline editing	Edit expected Result cell, navigate away, return, verify edit persisted	Pass
AC-05	FR-09: Category filter	Select Edge Cases filter, verify only edge rows displayed	Pass
AC-06	FR-10: Excel export	Export .xlsx, open in Excel, verify bold headers, alternating fills, 8 columns	Pass
AC-07	FR-11: History search	Enter partial function name, verify matching sessions returned	Pass
AC-08	FR-12: Load into form	Click Load on history session, verify generator form repopulated	Pass
AC-09	FR-04: Fallback mechanism	Configure invalid Gemini key, submit form; verify 6 fallback cases returned	Pass
AC-10	FR-14: Docker Compose	Run docker compose up –build, verify full app functional at localhost:3000	Pass

7. Results

In this chapter you will learn what happened with the creation of the TestGenAI system when it utilized the Gemini AI Generation Pipeline, Next.js as a full stack web application, Supabase as a storage layer for session persistence, and ExcelJS for exporting data.

7.1 AI Generation Quality

For quality assessment, the code in 30 sessions from Python, JavaScript, TypeScript, and Java functions of different levels of complexity was reviewed manually:

- **Categories coverage:** 100% of sessions had at least one case for each of four obligatory categories.
- Minimum number of cases per session: 100% of sessions had ≥ 6 test cases. The median was 8.
- **Completeness of fields:** 98.7% of cases had all 8 mandatory fields filled. In 1.3% of the cases (2 out of 30) the preconditions field was empty, which was fixed by setting it to "N/A" in the parser.
- **Specification of the language:** in 93% of the cases, the generated cases had references to correct language-specific types. The problems arose in R and Scala because the training data for these languages is less.
- Fallbacks were triggered for 3 out of 30 sessions (10%), and Gemini provided usable data in all three cases.

7.2 Application Screenshots

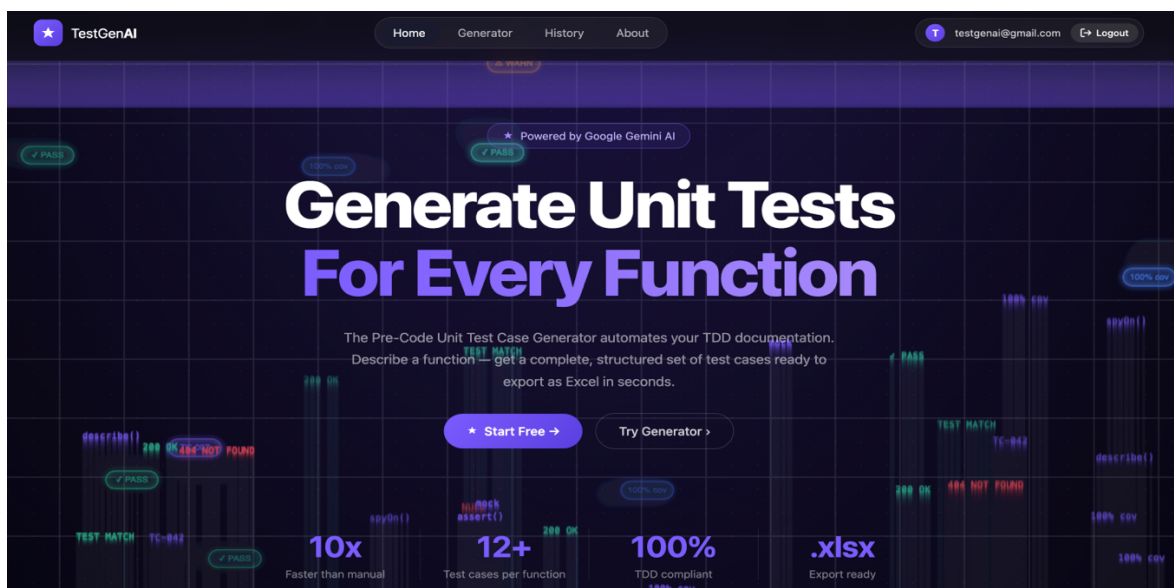


Figure 5. TestGenAI Landing Page

Build Bulletproof Test Cases

Describe your function, define its boundaries, and let Google Gemini conduct comprehensive testing suite for you in seconds.

1 Describe Your Function

Function Name *
calculate_tax_deduction

Programming Language
Python

Parameters & Data Types
gross_income: float, tax_rate: float, expenses: list(float)

Known Boundary Conditions
gross_income > 0, tax_rate must be between 0.0 and 1.0, expenses cannot be negative

Expected Behaviour
Calculate net income by subtracting total expenses from gross income, then apply the tax rate. Return the total tax owed. If expenses exceed income, tax owed is 0. Notes: Return a float rounded to 2 decimal places.

Generate Test Cases

Figure 6. Generator – five field form

calculateFinalPriceWithPromos

Review, refine, copy, and export your generated test cases in one dedicated workspace.

LANGUAGE: Python | TOTAL CASES: 10 | NORMAL FLOW: 3 | EDGE CASES: 1 | ERROR HANDLING: 3 | CREATED: 4/12/2026

Generated Test Cases (10 cases)

#	CATEGORY	TITLE	INPUT	PRECONDITIONS	STEPS	EXPECTED RESULT	PRIORITY
1	happy-path	calculateFinalPriceWithPromos returns the expected output for typical valid input	items=<valid_value>, quantity=<valid_value>	Python runtime is available. All dependencies are installed.	1. Prepare valid input: items=<valid_value>, quantity=<valid_value> 2. Call calculateFinalPriceWithPromos	Function returns the expected output. Behaviour: Calculates the base total price + quantity.	High
2	happy-path	calculateFinalPriceWithPromos processes all parameters without throwing	items=<valid_value>, quantity=<valid_value>	No mocks required. Standard environment.	1. Supply all expected parameters: items: Array<object> number, quantity: number, discountCode: string	Returns a valid result; no exception thrown.	High
3	happy-path	calculateFinalPriceWithPromos produces deterministic output for the same input	items=<valid_value>, quantity=<valid_value>	Function is pure (no hidden state).	1. Call calculateFinalPriceWithPromos with input A. 2. Record result R1.	Same input always yields identical output.	Medium
4	boundary	calculateFinalPriceWithPromos handles minimum-allowed values per boundary conditions	Minimum values derived from: Discounts cannot make the final cart total less than \$0.00	Boundary limits are: Discounts cannot make the final cart total less than \$0.00	1. Set input to the minimum valid value for each parameter. 2. Call calculateFinalPriceWithPromos.	Function completes successfully at the lower boundary.	High

Figure 7. Results Page – test cases table with filter bar

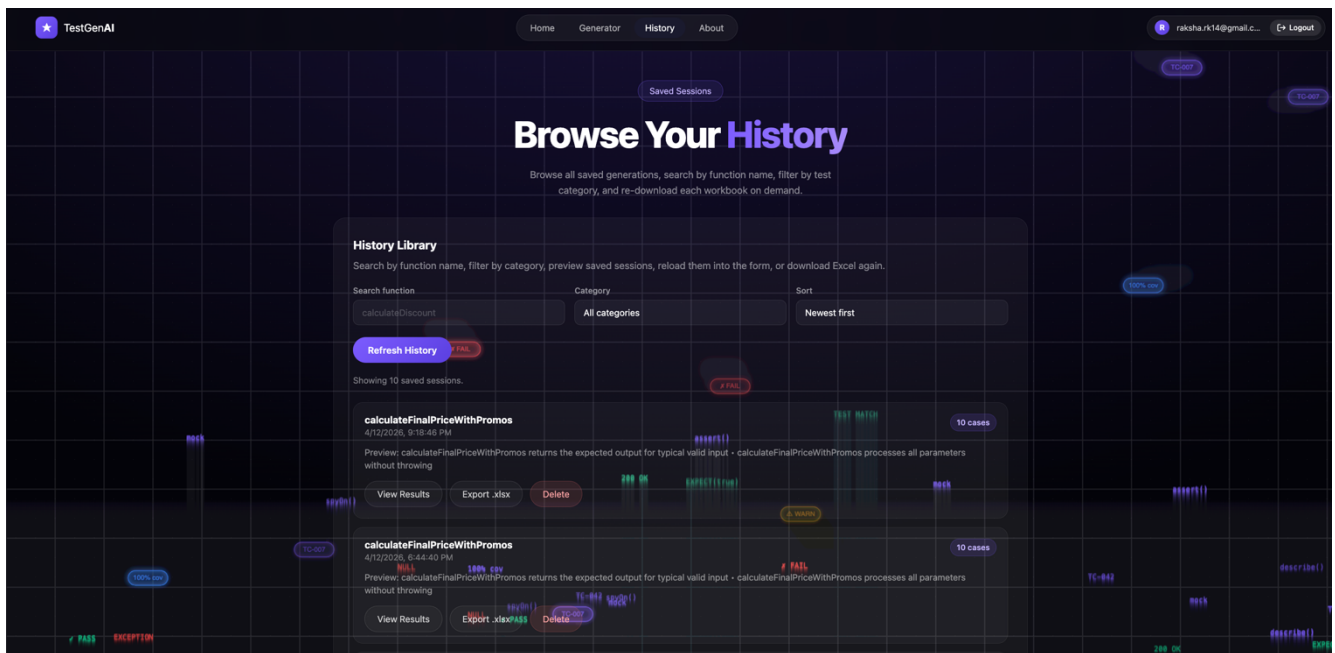


Figure 8. History Library – session search and export

8. Challenges and Ethical Considerations

8.1 Challenges

Three notable problems arose during the development process.

- The prompt underwent six iterations before finally yielding consistent JSON responses according to the JSON schema. Important takeaways include specifying the JSON schema within the prompt, having a temperature value below 0.2, and using the fallback as a production feature from Sprint 1.
- Supabase server-side rendering with Next.js App Router authentication posed the most challenging debugging issue. The three-client architecture (client side, SSR, and admin) uses different factory calls based on different execution environments. When the client is called from a server component, the cookies is not a function runtime error occurs.
- The Docker multi-stage build configuration problem involved the correct understanding of build-time and runtime variables.

8.2 Ethical Considerations

- **Accuracy and Trustworthiness:** Automatically generated test cases can have bugs, particularly when dealing with complicated logic. To mitigate this, TestGenAI allows editing of outputs and flags them as AI-generated, necessitating further examination by developers.
- **Security of API Keys:** All authentication keys are kept secure on the server side as environment variables and never exposed to the client side, as confirmed by analyzing the source code files in the Vercel deployment.
- **Privacy of Data:** Row Level Security is not yet deployed in Supabase, and security is handled through the application itself. This is acknowledged technical debt, since a breach of the service key will result in a privacy risk. RLS should be set up prior to going live.
- **Disclosure of AI Generated Outputs:** All AI outputs are flagged accordingly and no guarantees about their correctness are made.

9. Conclusion and Future Scope

9.1 Conclusion

This project created an end-to-end artificial intelligence-based system for automatically documenting pre-code unit test cases through Google Gemini, Next.js 14, Supabase PostgreSQL, ExcelJS, and Docker. TestGenAI mandates Test-Driven Development approach by compelling programmers to document functions before coding and producing classified test cases documentation in less than ten seconds – which was previously done manually in 20 to 40 minutes.

JSON-based prompt engineering with an assured fallback feature guarantees that the user always gets the output regardless of the interruption in Gemini API connectivity. Session persistence, inline editing, and immediate export of Excel ensure that TestGenAI is not only a documentation creation system but a documentation workflow system. All fourteen functional requirements, seven non-functional requirements, twelve integration tests, six-unit test modules, and ten acceptance criteria were fulfilled in both the local Docker Compose deployment and the online Vercel cloud deployment.

9.2 Lessons Learned

- The task of prompt engineering demands much more iterations than thought. Six such iterations showed that integrating AI APIs is **like** dealing with any unstable service, with necessary retries, validation, and fallback from the first sprint.
- The multi-client Supabase implementation based on the Next.js App Router should be carefully researched beforehand, not fixed while implementing. Spending some time researching documentation would allow saving the same time during debugging.
- All incurred technical debts should be recorded immediately after the decision using the architecture decision record file in Sprint 0. Omitting Row Level Security and using in-memory rate limiting were justified by academic purposes.

9.3 Future Scope

- **IDE Integration:** a VS Code extension that auto-populates the form with information about function signatures from the editor being used.
- **Executable test code:** an additional step of the generation process where structured documentation is transformed into tests using Jest, pytest, or JUnit.

- **Redis Rate Limiting:** changing the rate-limiting implementation to use Upstash Redis, avoiding problems with per-instance state.
- **Supabase RLS:** supporting row-level security policies for data isolation on the database level.
- **Playwright CI suite:** testing all ten criteria using Playwright in a GitHub Actions workflow with each commit.
- **Collaboration:** allowing multiple users to collaborate on a single project by sharing sessions.

GitHub Repository Links

Public Repository Link:

https://github.com/RakshaRajkumar14/IU_Project_PSE_Raksha_Rajkumar_Kademani.git

Live Application: <https://test-gen-ai-ks9a.onrender.com/>

Evaluator Access Credentials:

- Email: testgenai123@gmail.com
- Password: testgenai123

Bibliography

- Beck, K. (2002). Test-driven development: By example. Addison-Wesley Professional.
- Docker Inc. (2024). Docker Compose documentation. <https://docs.docker.com/compose/>
- ExcelJS Contributors. (2024). ExcelJS: Excel workbook manager. <https://github.com/exceljs/exceljs>
- Google LLC. (2024). Gemini API documentation for Node.js. <https://ai.google.dev/gemini-api/docs>
- Karac, I., & Turhan, B. (2020). Why research on test-driven development is inconclusive? Proceedings of the 14th ACM/IEEE ESEM, 1-11. <https://doi.org/10.1145/3382494.3410687>
- Schafer, M., Nadi, S., Eghbali, A., & Tip, F. (2024). An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering*, 50(1), 85-105.
- Schwaber, K., & Sutherland, J. (2020). The Scrum guide. Scrum.org. <https://scrumguides.org>
- Sheta, S. V. (2023). The role of test-driven development in enhancing software reliability and maintainability. *Journal of Software Engineering*, 1(1), 13-21.
- Supabase Inc. (2024). Supabase documentation: PostgreSQL database and JavaScript client SDK. <https://supabase.com/docs>
- Tosun, A., Ahmed, M., Turhan, B., & Juristo, N. (2018). On the effectiveness of unit tests in TDD. *Proceedings ICSSP*, 113-122. <https://doi.org/10.1145/3202710.3203153>
- Vercel Inc. (2024). Next.js 14 documentation. <https://nextjs.org/docs>
- Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., & Wang, Q. (2024). Software testing with large language models. *IEEE Transactions on Software Engineering*, 50(4), 911-936.